

PluralTree

Hyperbolic Tree-GRU with Gated Knowledge Injection: Codebase, Architecture, and Experiment Guide

A concrete walkthrough of the model, math, and experiments
Generated 2026-05-31

1. Overview

PluralTree learns hierarchical knowledge-graph embeddings in the Poincare ball. A Hyperbolic Tree-GRU encodes a rooted tree bottom-up on the manifold; Gated Knowledge Injection (GKI) blends external knowledge into each node. The model is trained by link prediction on CulturalBench and evaluated with filtered MRR / Hits@k. This document is deliberately concrete: it gives the actual operations, tensor shapes, and hyperparameters used in the code.

Concrete dimensions for the default CulturalBench run

N (entities)	= 1284	(1 world + 11 regions + 43 countries + ~1229 practices)
num_relations	= 3	(practiced_in, located_in, part_of)
d_text (input)	= 768	(all-mpnet-base-v2; 384 for all-MiniLM-L6-v2)
d_hidden	= 128	(hidden size = relation-embedding size)
curvature c	= 1.0	(ball radius $1/\sqrt{c} = 1$)
train / val / test	= 1037 / 178 / 180	triples
n_negative	= 10	margin = 1.0 batch_size = 128

2. Why Hyperbolic Geometry

A tree with branching factor b has $\sim b^d$ nodes at depth d . Euclidean volume grows only polynomially with radius, so embedding a tree there forces distortion. Hyperbolic volume grows exponentially with radius, matching the tree, so trees embed with low distortion in few dimensions. In the Poincare ball, the origin acts as the abstract root and the boundary as specific leaves, so a node's radius $\rho = \sqrt{c} * ||h||$ encodes its depth. This is the geometric fact the whole architecture exploits.

3. Architecture in Detail

3.1 Manifold operations (pluraltree/manifolds/poincare.py)

Every state lives on the ball; all of the following are the exact operations used. $\langle x, y \rangle$ is the dot product, $||x||$ the L2 norm.

Mobius addition (translation on the ball):

$$x (+) y = \frac{(1 + 2\langle x, y \rangle + c||y||^2) x + (1 - c||x||^2) y}{1 + 2\langle x, y \rangle + c^2 ||x||^2 ||y||^2}$$

Exp map at origin (tangent $\rightarrow$ ball): $\exp_0(v) = \tanh(\sqrt{c}||v||) / (\sqrt{c}||v||) * v$

Log map at origin (ball $\rightarrow$ tangent): $\log_0(x) = \operatorname{artanh}(\sqrt{c}||x||) / (\sqrt{c}||x||) * x$

Geodesic distance: $d(x, y) = (2/\sqrt{c}) * \operatorname{artanh}(\sqrt{c} ||(-x) (+) y||)$

Conformal factor: $\lambda(x) = 2 / (1 - c||x||^2)$

Numerical safety (math_utils.py): norms are floored (MIN_NORM), artanh is clamped below 1, and every result is projected just inside the boundary so points never reach radius $1/\sqrt{c}$.

3.2 Child aggregation (tree_gru/aggregation.py)

Children are combined with the Einstein / Mobius weighted midpoint, which is the proper 'average' of points on the ball. With K padded children $h_children$ of shape (K, B, d) and weights w of shape $(K, B, 1)$:

```
gamma = sum_k [ w_k * lambda(x_k) * x_k ] / sum_k [ w_k * (lambda(x_k) - 1) ]
result = project(gamma) # midpoint on the ball, shape (B, d)
```

Weights come from attention: $\text{score}_k = W_a(\log_0(h_k))$ with W_a a $(d \rightarrow 1)$ linear; padded children are masked to $-\infty$, then softmax over k . (A uniform-weight mode exists too.) Masked/padded children get weight 0 and contribute nothing, which is what makes the level-batched encoder exact.

3.3 The Hyperbolic Tree-GRU cell (tree_gru/cell.py)

Each node's input feature x is the frozen text embedding passed through input_proj (a $768 \rightarrow 128$ linear); x lives in tangent space. The children summary h_agg lives on the ball. The cell maps h_agg to tangent space, runs a standard GRU there, and maps back. Shapes shown for a batch of B nodes, $d_hidden = 128$.

```
h_agg = mobius_midpoint(children) # (B,128) on ball
h_t   = log0(h_agg)              # (B,128) tangent space
```

```

z      = sigmoid( W_z [h_t ; x] )          # update gate   (B,128)
r      = sigmoid( W_r [h_t ; x] )          # reset gate    (B,128)
n      = tanh(    W_n [r*h_t ; x] )        # candidate     (B,128)
h_t'   = (1 - z) * h_t + z * n             # GRU mix       (B,128)

h_v    = exp0(h_t')                        # (B,128) back on the ball

```

```
W_z, W_r, W_n : Linear(256 -> 128)  ([h_t ; x] concatenates 128+128)
```

Leaves (practices) have no children, so h_{agg} is the origin ($h_t = 0$) and $h_v = \exp_0(z * \tanh(W_n[0 ; x]))$ depends only on the leaf's own text. The same three weight matrices are reused at every node — there is no per-node embedding table. Running the gates in tangent space avoids defining sigmoid/tanh on the curved manifold.

3.4 Gated Knowledge Injection and the depth-aware gate

GKI mixes the structural state h with a knowledge vector k . k is the node's text embedding sent through a `ProjectionLayer` (Linear + LayerNorm + $\exp_0$), so k is a point on the ball, shape (B,128). The depth-aware gate (combined/depth_aware_gate.py) appends the radius ρ :

```

rho = sqrt(c) * ||h||                      # (B,1) depth signal
g   = sigmoid( W_g [ log0(h) ; log0(k) ; rho ] + b ) # (B,128) gate, W_g: 257->128
h'  = mobius_midpoint( h, k ; weights = [1 - g, g] ) # (B,128) blended, on ball

```

g near 0 keeps h (trust structure); g near 1 adopts k (trust knowledge). A second head $\beta = \text{sigmoid}(w_{\beta} * \rho + b_{\beta})$ blends a broad vs. precise knowledge source by depth (active once a distinct second source exists). Injection point is configurable: PRE_AGGREGATION, POST_AGGREGATION (default), POST_GRU, or DUAL — swept in the A1 ablation.

3.5 Curriculum: the gate-bias schedule (combined/knowledge_schedule.py)

The bias b in the gate is pinned by a 3-phase schedule on the optimizer step:

```

step < warmup1          : b = -2.0          sigmoid(-2.0) ~ 0.12 (nearly closed)
warmup1 <= step < warmup2 : b ramps -2.0 -> 0 linearly
step >= warmup2         : b = 0.0           unbiased (neutral 0.5)

```

Defaults used: $warmup1 = 400$, $warmup2 = 1600$. With ~ 9 steps/epoch, gates become unbiased near epoch 17, so 300 epochs leaves a long stretch of training with knowledge available. Starting closed forces the model to learn structure before it can lean on knowledge.

3.6 Scoring head (training/scoring.py)

TransE-style translation on the ball. Each relation is a learned point r on the ball (geoopt ManifoldParameter, shape (3,128), initialised uniform in $[-0.001, 0.001]$).

```

translated = h_s (+) r_relation          # Mobius addition, (B,128)
score      = - d( translated, h_o )      # negative geodesic distance, (B,)

```

Higher score = more plausible triple. After translating by the relation, prediction is exactly nearest-neighbour search in hyperbolic distance.

3.7 Negatives and the training loss

Negatives corrupt the OBJECT with a same-type entity (data/negative_sampler.py). Type constraints: practiced_in -> the 43 countries, located_in -> the 11 regions, part_of -> {World}. n_negative = 10 per positive; 'filtered' sampling skips known true triples. The loss is a margin ranking term over all B*K negatives plus a small gate-sparsity penalty:

```
L_lp      = mean( relu( margin + score_neg - score_pos ) )      # margin = 1.0
L_sparse  = mean( | W_g.weight | )                               # proxy for gate usage
L         = L_lp + 0.01 * L_sparse
```

Note: the sparsity term penalises gate-weight magnitude as a proxy; a true L0/hard-concrete gate (see GATING.md) would penalise activations directly.

3.8 Evaluation (evaluation/link_prediction.py)

For each test triple (s, r, o): score s against every type-valid candidate object (e.g. all 43 countries for practiced_in), remove other known positives (filtered setting), and rank the true o. Aggregate over triples:

```
rank      = 1 + #{ candidate c : score(s,r,c) > score(s,r,o), c not a known positive }
MRR       = mean( 1 / rank )
Hits@k    = fraction of triples with rank <= k          for k in {1, 3, 10}
```

3.9 Optimizer (utils/riemannian_optim.py)

Parameters are split by type. Relation embeddings (ManifoldParameter) use geoopt.RiemannianAdam at lr_manifold = 1e-2 for curvature-aware updates that stay on the ball; all Euclidean weights (input_proj, aggregator, GRU, gate) use ordinary Adam at lr = 1e-3. Gradients are norm-clipped. The encoder is re-run every batch (h is recomputed, never cached), so gradients are exact for the current weights.

3.10 The level-batched encoder (combined/gki_tree_encoder.py)

The reference path walks nodes one at a time: 1284 tiny cell calls per encode, x ~2700 encodes per run. Cost is dominated by kernel-launch overhead, not arithmetic. The fast path groups nodes by height (leaf = 0, height(v) = 1 + max child height). Every child has strictly smaller height, so a whole level is mutually independent and runs as ONE padded, masked, batched call:

```
height 0: ~1229 practices    -> 1 batched leaf step
height 1: 43 countries       -> 1 batched cell call (children padded to max_k)
height 2: 11 regions         -> 1 batched cell call
height 3: 1 world            -> 1 batched cell call
```

```
per encode: 1284 cell calls -> 4 cell calls
```

The padding plan is built once and cached (the tree is static). Verified identical to the sequential path: float64 difference ~1e-13 (machine epsilon). Measured speedup: 55x on CPU, larger on GPU.

4. The Dataset: CulturalBench

kellycyy/CulturalBench (Easy split): multiple-choice cultural questions. PluralTree keeps only the geography and the question text. The graph is a single rooted tree:

```
World (1)
```

```
+ - Region (11): East_Asia, Western_Europe, Africa, ...
+ - Country (43): China, Japan, France, Brazil, ...
+ - Practice (~1229): one node per unique question
```

Relations: practiced_in (practice->country), located_in (country->region), part_of (region->world). Structural triples are always visible; practice triples are split 80/10/10. Each entity's description is embedded with a frozen sentence-transformer used as BOTH the node feature x and the knowledge vector k — a current limitation: because they share a source, the gate has no genuinely new information to inject yet (roadmap B2 adds a distinct source).

Leakage guard: the graph is built from the questions, so any LLM evaluation must exclude a test question's own node and all val/test practices.

5. Project Layout (key modules)

- pluraltree/manifolds/poincare.py - Mobius ops, exp/log maps, distance, midpoint
- pluraltree/tree_gru/ - HyperbolicTreeGRUCell + ChildAggregator
- pluraltree/gki/ - EuclideanGate / HyperbolicGate + GKIInjector
- pluraltree/combined/ - depth-aware gate, GKI cell, encoder, schedule
- data/culturalbench.py - build CulturalGraph + frozen text embeddings
- data/negative_sampler.py - type-constrained corruption + filtered ranking
- training/ - trainer, scoring head, losses
- evaluation/link_prediction.py - filtered MRR / Hits@{1,3,10}
- scripts/train.py - CLI entry point (banner + RESULT summary line)
- jobs/ - SLURM scripts incl. the A1 ablation set

6. Experiments and Their Purpose

Principle: validate each component before building on it. An early run showed opening the gates REDUCED MRR, so GKI is not yet proven to help — making the ablations urgent.

6.1 A1 ablations (currently running)

- --no_gki: pure Tree-GRU. Baseline vs. this is the central test — does injection help at all?
- --gate_type plain|depth_aware: does conditioning the gate on radius ρ matter?
- --injection pre_agg|post_agg|post_gru|dual: where should knowledge enter?
- Euclidean vs. hyperbolic (planned): does the ball beat flat space here? (needs a Euclidean manifold.)

Each run prints one grep-able line: RESULT | config | best_val_mrr | test_mrr h@1 h@3 h@10.

6.2 Roadmap A-E

- A Validation: ablations, rho-vs-depth, baselines (TransE/RotatE, frozen-encoder NN).
- B Architecture: stronger encoders, a distinct 2nd knowledge source, learnable c , top-down pass.
- C Scale/generalisation: larger/deeper KGs, inductive eval on held-out subtrees, DAG structure.

- D LLM integration: hierarchical retrieval, traversal-as-tool, soft-prompt injection, uncertainty prompting.
- E Plurality/distributions: Wrapped-Normal embeddings, Pluralistic Leaf Existence, and their synthesis.

7. End Goal: KG-Augmented LLM Reasoning

Target: integrate the KG with a fine-tunable open 7B (LoRA) to aid CulturalBench QA, climbing a coupling ladder so each step justifies the next:

- 1. Hierarchical retrieval (walk World->...->Practice) injected as text, vs. flat RAG.
- 2. Embedding-guided traversal exposed as an LM tool (hyperbolic NN + tree paths).
- 3. Soft-prompt injection: project h_v into the LM token space, train with LoRA.
- 4. Cross-attention fusion (GreaseLM-style) once the KG signal is proven.

Honest metric: KG-grounded LM vs. LoRA-only LM with no KG. Guardrail: never retrieve the target question's own node or any val/test practice.

8. Research Frontiers

Single-parent today because the loader hardcodes one parent and the bottom-up encoder gives one embedding per node. A plain DAG (multiple parents, one embedding) is mostly a data change — the encoder already tolerates it. Pluralistic Leaf Existence (a distinct embedding per parent context) needs top-down flow or per-edge materialisation. Deeper still: hierarchy need not be specified — soft/learned attachment becomes a well-posed problem once multi-benchmark data introduces attachment ambiguity.

9. Running the Code

```
# local
python scripts/train.py --d_hidden 128 --n_epochs 300 --device cuda \
    --embed_model all-mpnet-base-v2

# SLURM (Narval) - A1 ablation matrix
for j in baseline no_gki plain_gate pre_agg post_gru dual; do
    sbatch jobs/job_a1_$j.sh
done
```

Compute nodes are offline: pre-download dataset + encoder on the login node; job scripts set HF_HUB_OFFLINE / TRANSFORMERS_OFFLINE to load from cache, and PYTHONUNBUFFERED=1 keeps the .out log and intact if a job is killed. pyarrow comes from the 'arrow' module (load before activating the venv), not pip.

Companion docs in the repo: *README.md*, *EXPERIMENTS.md*, *RELATED_WORK.md*, *GATING.md*.